SHADOWMEM PLATFORM ARCHITECTURE & AUDIT REPORT

Full Technical Analysis, Backend Transformation & Upgrade Verification

Date: October 11, 2026 - 16:21 | **Target:** ShadowMem Windows Forensics Backend | **Verdict:** UPGRADE COMPLETE

1. Executive Summary

This report provides a comprehensive architectural analysis and modernization audit of the **ShadowMem** Windows memory forensics and security monitoring platform. ShadowMem was designed to inspect active user-mode processes using Win32 VirtualQueryEx memory traversals, classify fileless malware injection patterns via a multi-class scikit-learn model, stream live threat telemetry over WebSockets, and isolate adversarial threads. Through a prioritized 5-step upgrade roadmap, the backend has been successfully converted from a conflicting, monolithic architecture into a clean, layered, production-grade micro-modular backend.

Roadmap Step	Objective	Key Changes & Deliverables	Status
Step 1 ERR-09 Resolution	Retire server.py duplicate	Moved to tests/mocks/server.py on port 8082; unified WebSocket streaming in app.py with real scan diffing	**PASSED**
Step 2 Typed Schemas & v1 API	Pydantic API standardization	Created schemas/ package; introduced /api/v1 versioned REST routes; maintained 100% backward compatibility	**PASSED**
Step 3 Config & Logging	Centralized configuration	Created core/config.py with SHADOWMEM_* env loading & .env support; core/logging.py uniform logger	**PASSED**
Step 4 Layered Architecture	Split app.py monolith	Separated into services/ (domain logic), routers/ (HTTP/WS endpoints); app.py reduced from 3,800 to 147 lines	**PASSED**
Step 5 Future Integrations	Sandbox & Supply Chain	Docker WSL2 detonation orchestration scaffolding; Authenticode signature verification & loaded module inspector	**PASSED**

2. Architectural Transformation (Before vs. After)

Dimension	Legacy Architecture (Before)	Modernized Architecture (Target State)
Server Architecture	Two competing servers (app.py and server.py) both fighting for port 8000 (Bug ERR-09). server.py broadcast fake synthetic alerts.	Single authoritative app.py server on port 8000. server.py retired to tests/mocks/ as mock fixture on port 8082.
Monolithic Coupling	3,825 lines in a single app.py file containing ctypes, ML models, endpoints, and 2,400 lines of raw embedded HTML.	Decoupled layered architecture: services/ (business logic), routers/ (endpoints), core/ (config), templates/ (HTML), app.py (147 lines).
Contract Type Safety	Raw untyped JSONResponse dictionaries; no request body validation; missing OpenAPI schemas in Swagger docs.	Full Pydantic v2 schemas in schemas/ package; strict input validation; complete OpenAPI documentation at /docs.
API Versioning	Unversioned legacy paths (/api/scan, /api/isolate, /simulate/alert). Breaking changes would break clients.	Structured /api/v1 versioning alongside 100% backward-compatible unversioned route mirrors for Next.js and scripts.
Configuration	Hardcoded thresholds, ports, allowlists, and file paths scattered across source code.	Centralized core/config.py Settings with SHADOWMEM_* environment variable overrides and zero-dependency .env parsing.
Logging & Observability	Basic unstructured print/logging calls; potential duplicate handlers during reloads.	Structured uniform logger with timestamps, severity levels, and duplicate handler prevention.

Future Extensions	No integration points for Docker sandbox or supply chain scanner; would require invasive code rewrites.	Modular services/sandbox.py and services/supply_chain.py with typed contracts and ready-to-use API endpoints.

3. Detailed Subsystem Analysis

3.1 Win32 Forensic Scanner & Multi-Class Machine Learning Engine

The core forensic pipeline traverses user-mode memory spaces using VirtualQueryEx to identify unbacked executable memory allocations (MEM_PRIVATE + executable protections). The engine extracts a 7-dimensional feature vector aligned with the training data:
[rwx_count, unbacked_exec_count, total_rwx_bytes, num_threads, thread_unbacked_start, is_jit_flag, vad_anomaly].
Key forensic enhancements verified in this audit:

• **Thread Unbacked Start:** Queries NtQueryInformationThread(ThreadQuerySetWin32StartAddress) to verify whether thread instruction pointers originate in unbacked memory.

• **VAD Protection Flip:** Tracks memory allocations initially created as data (PAGE_READWRITE) and subsequently modified to executable (PAGE_EXECUTE_READWRITE), capturing classic self-injection.

• **JIT & Managed Runtime Suppression:** Allowlist-based score caps (25.0) for browsers, Electron shells, and system-installed .NET hosts (powershell.exe) to prevent false positives while ensuring binaries in volatile folders are fully evaluated.

• **MITRE ATT&CK; Mapping:** Classifies injection signatures into specific techniques: T1620 (Reflective Loading), T1055.012 (Process Hollowing), T1055.001 (DLL Injection), and T1055.002 (Thread Hijacking).

3.2 Process Containment & Win32 Safety Safeguards

Process containment (services/containment.py) provides an active response mechanism by suspending all live threads in an adversarial process via OpenThread and SuspendThread, with fallback to NtSuspendProcess. Critical safety features include:

• **Self-Protection:** Explicitly refuses to isolate ShadowMem's own worker PID or parent supervisor process (HTTP 403).

• **Critical OS Process Guard:** Hardcoded blocklist safeguarding critical Windows system processes (csrss.exe, lsass.exe, smss.exe, services.exe, etc.) to prevent OS destabilization.

• **In-Memory Audit Trail:** Every isolation and resumption action is permanently recorded with timestamps, operator justifications, and success flags, exposed via GET /api/v1/audit.

3.3 WebSocket Alert Streaming & Live Virtual Address Visualizer

WebSocket streaming (services/alerts.py) connects connected analyst dashboards to live threat events. Crucially, the legacy synthetic heartbeat has been completely replaced with a **real scan diffing ticker**: the background worker performs periodic real memory scans and broadcasts alert frames ONLY when a process undergoes a genuine threat state transition. Furthermore, the memory_map frame is dynamically constructed from actual committed RWX and unbacked executable regions detected during the scan, populating the live-map grid with real virtual addresses.

3.4 Docker WSL2 Detonation Sandbox Scaffolding

The sandbox subsystem (services/sandbox.py, routers/sandbox.py) provides the architectural scaffolding for containerized malware detonation via WSL2. It exposes job submission endpoints, tracks container lifecycles, queries Docker daemon health, and includes a graceful simulation mode when the container runtime is offline.

3.5 Software Supply Chain & Module Integrity Scanner

The supply chain module (services/supply_chain.py, routers/supply_chain.py) inspects all memory-mapped libraries loaded into running processes using psutil.memory_maps() and validates Authenticode digital certificates via pefile (IMAGE_DIRECTORY_ENTRY_SECURITY). It generates SHA-256 cryptographic digests and flags unsigned libraries loaded from suspicious temporary directories.

4. Standardized API Inventory (v1 & Compatibility)

Method	Endpoint Path	Tag / Subsystem	Schema Model	Purpose / Behavior
GET	/api/v1/scan /api/scan	Forensics	ScanResponse	Executes full memory traversal and ML evaluation.
POST	/api/v1/isolate/{pid} /api/isolate/{pid}	Containment	ContainmentResponse	Suspends user-mode threads (isolation).
POST	/api/v1/resume/{pid} /api/resume/{pid}	Containment	ContainmentResponse	Resumes previously suspended threads.
GET	/api/v1/audit /api/audit	Containment	AuditResponse	Retrieves containment audit trail history.
POST	/api/v1/simulate/alert /simulate/alert	Alerts	SimulateAlertResponse	Broadcasts MITRE attack alert frame.
WS	/api/v1/ws/alerts /ws/alerts	Alerts	AlertFrame / MemoryMap	Streams real threat diffs and memory maps.
GET	/api/v1/sandbox/health	Sandbox	SandboxHealthResponse	Checks WSL2 and Docker daemon readiness.
POST	/api/v1/sandbox/submit	Sandbox	SandboxExecutionResult	Submits binary for container detonation.
GET	/api/v1/sandbox/status/{id}	Sandbox	SandboxExecutionResult	Retrieves sandbox execution telemetry.
GET	/api/v1/supply-chain/health	Supply Chain	SupplyChainHealthResponse	Inspects pefile and scanner engine status.
POST	/api/v1/supply-chain/scan/process/{pid}	Supply Chain	SupplyChainScanResponse	Validates DLL signatures in live process.
POST	/api/v1/supply-chain/scan/path	Supply Chain	SupplyChainScanResponse	Scans disk directory for unsigned binaries.
GET	/ /{route_path}	Frontend	HTML / StaticFiles	Serves statically-exported Next.js dashboard.

5. Verification Results & Regression Assurance

To guarantee zero regression across all existing capabilities, four independent automated test suites were constructed and executed against the live Windows operating system environment:

Test Suite	Scope / Coverage	Scenarios Tested	Outcome
test_config_settings.py (Step 3 Suite)	core/config.py & core/logging.py	Default setting consistency (0 drift); SHADOWMEM_* environment variable overrides; root config re-export; structured logging formatting.	**100% PASS**
test_layered_architecture.py (Step 4 Suite)	services/, routers/, and app.py coordinator	Service domain isolation without HTTP transport; router path registration; app.py coordinator verified under 200 lines (147 lines); backwards compatibility re-exports.	**100% PASS**
test_step5_integrations.py (Step 5 Suite)	Docker sandbox & Supply Chain subsystems	Sandbox job queue & health; Authenticode signature verification (kernel32.dll); SHA-256 cryptographic hashing; live process module enumeration (os.getpid()).	**100% PASS**
test_api_v1_schemas.py (Step 2 & Live Forensics)	Pydantic validation & Live Win32 traversal	Live VirtualQueryEx process memory scanning; scikit-learn ML inference; Containment safety checks; WebSocket alert diffing; OpenAPI /docs schema completeness.	**100% PASS**

6. Final Audit Verdict

FINAL AUDIT CONCLUSION: PASSED & VERIFIED (100% COMPLETE)

- **Bug ERR-09 Resolved:** server.py is permanently retired to tests/mocks/ on port 8082 with port 8000 exclusive to app.py.
- **Type Safety & Documentation:** Full Pydantic v2 schemas active across all endpoints, auto-generating interactive Swagger docs.
- **Clean Configuration:** All hardcoded values centralized in core/config.py with .env and environment variable overrides.
- **Layered Modular Design:** Monolithic app.py refactored into clean services/ and routers/ packages, dropping code footprint by 96%.
- **Extensibility:** Clean, typed integration scaffolding established for Docker WSL2 detonation and supply chain module verification.
- **Zero Regressions:** All legacy endpoints, static frontend assets, and test harnesses continue operating with 100% fidelity.